Module 00: Getting Started with Laravel

Duration: 3-5 days | **Level:** Beginner | **Prerequisites:** Basic computer skills

Learning Objectives

By the end of this module you will:

- Understand what Laravel is and when to use it
- Install PHP, Composer, and a local development environment
- Create your first Laravel project
- Run the development server and Artisan commands
- Navigate the project folder structure

1. What Is Laravel?

Laravel is a **PHP web application framework** that provides:

Feature	Benefit
MVC architecture	Separates logic, data, and presentation
Eloquent ORM	Database work without raw SQL
Blade templating	Clean, reusable HTML views
Built-in auth, queues, mail	Production features out of the box
Massive ecosystem	Packages for payments, search, admin panels

Laravel follows the **convention over configuration** philosophy - sensible defaults so you write less boilerplate.

2. Prerequisites

Required Software

Tool	Minimum Version	Purpose
PHP	8.2+	Runtime (you have 8.2.12 [x])
Composer	2.x	PHP dependency manager
Node.js	18+	Frontend asset compilation (Vite)
Git	Any recent	Version control
Database	SQLite/MySQL/PostgreSQL	Data storage

Windows Setup (XAMPP - your current setup)

You already have PHP via XAMPP at `C:\xampp\php\php.exe`. Ensure these PHP extensions are enabled in `php.ini`:

```
extension=curl
extension=fileinfo
extension=mbstring
extension=openssl
extension=pdo_mysql
extension=pdo_sqlite
extension=tokenizer
extension=xml
```

Verify:

```
php -v
composer --version
php -m | findstr pdo
```

Optional but Recommended

- **Laragon** or **Herd** - simpler Laravel dev environments on Windows
- **VS Code** or **PhpStorm** with PHP Intelephense
- **TablePlus** or **DBeaver** - database GUI

---

3. Creating a Laravel Project

```
composer create-project laravel/laravel my-app
cd my-app
cp .env.example .env        # Windows: copy .env.example .env
php artisan key:generate
```

What Composer Does

1. Downloads `laravel/laravel` (the application skeleton)
2. Resolves all dependencies into `vendor/`
3. Generates `composer.lock` for reproducible installs

Note: Laravel 13 requires PHP 8.3+. With PHP 8.2 you get Laravel 12 - fully capable for learning and production.

---

4. Environment Configuration (`.env`)

The `.env` file holds secrets and environment-specific settings:

```
APP_NAME=TaskForge
APP_ENV=local
APP_KEY=base64:...          # Generated by key:generate
APP_DEBUG=true
APP_URL=http://localhost

DB_CONNECTION=sqlite        # Easiest for learning
# DB_CONNECTION=mysql
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=taskforge
# DB_USERNAME=root
# DB_PASSWORD=

QUEUE_CONNECTION=database
CACHE_STORE=database
SESSION_DRIVER=database
```

SQLite Quick Setup (TaskForge default)

```
touch database/database.sqlite   # Windows: type nul > database\database.sqlite
php artisan migrate
```

---

5. Running the Application

```
# Start development server
php artisan serve
```

```
# -> http://127.0.0.1:8000

# Watch logs in real time (Laravel 11+)
php artisan pail

# Compile frontend assets (separate terminal)
npm install && npm run dev
```

Health Check

Laravel 11+ registers /up automatically. Visit http://localhost:8000/up - should return {"status":"ok"}.

6. Artisan - The Laravel CLI

Artisan is Laravel's command-line interface:

```
php artisan list                    # All commands
php artisan make:model Task -mfc    # Model + migration + factory + controller
php artisan migrate                 # Run migrations
php artisan migrate:fresh --seed    # Reset DB + seed data
php artisan route:list              # Show all routes
php artisan tinker                  # Interactive REPL
php artisan test                    # Run PHPUnit tests
php artisan queue:work              # Process queued jobs
php artisan schedule:work           # Run scheduler locally
```

Tip: Use php artisan make: with flags to scaffold multiple files at once.

7. Project Folder Structure

```
taskforge/
|-- app/
|   |-- Http/
|   |   |-- Controllers/     # Handle HTTP requests
|   |   |-- Middleware/      # Filter requests (auth, etc.)
|   |   `-- Requests/        # Form validation classes
|   |-- Models/              # Eloquent models (database entities)
|   |-- Policies/            # Authorization rules
|   |-- Jobs/                # Queued background tasks
|   |-- Events/              # Domain events
|   |-- Listeners/           # React to events
|   `-- Providers/           # Service registration & bootstrapping
|-- bootstrap/
|   `-- app.php              # Application bootstrap (middleware, routes)
|-- config/                  # Configuration files
|-- database/
|   |-- migrations/          # Database schema versions
|   |-- seeders/             # Test/demo data
|   `-- factories/           # Fake data generators
|-- public/                  # Web root (index.php)
|-- resources/
|   |-- views/               # Blade templates
|   |-- css/                 # Source stylesheets
|   `-- js/                  # Source JavaScript
|-- routes/
|   |-- web.php              # Web routes (sessions, CSRF)
|   |-- api.php              # API routes (stateless, tokens)
|   `-- console.php          # Artisan commands & scheduler
|-- storage/                 # Logs, cache, uploads
|-- tests/                   # Automated tests
`-- vendor/                  # Composer packages (don't edit)
```

8. The Request Lifecycle (Preview)

```
HTTP Request
-> public/index.php
-> bootstrap/app.php
-> HTTP Kernel (middleware pipeline)
-> Router (match route)
-> Controller action
-> Response (view, JSON, redirect)
-> HTTP Response
```

Module 02 covers this in depth.

---

9. Hands-On: Set Up TaskForge

```
cd taskforge
copy .env.example .env
php artisan key:generate

# Create SQLite database file
echo. > database\database.sqlite

# Run migrations and seed demo data
php artisan migrate --seed

# Start server
php artisan serve
```

Login credentials after seeding:
- **Admin:** `admin@taskforge.test` / `password`
- **Member:** `member@taskforge.test` / `password`

---

10. Exercises

1. Change `APP_NAME` in `.env` to your name and confirm it appears in the browser tab.
2. Run `php artisan route:list` and identify the `/up` health route.
3. Open `php artisan tinker` and run `App\Models\User::count()`.
4. Create a new route in `routes/web.php` that returns your name as JSON.
5. Switch `DB_CONNECTION` to MySQL (if available) and re-run migrations.

---

11. Self-Check Quiz

1. What is the purpose of `APP_KEY`?
2. Why should `.env` never be committed to Git?
3. What command generates a new migration file?
4. What folder is the web server's document root?
5. What is the difference between `migrate` and `migrate:fresh`?

Answers

1. Encrypts sessions, cookies, and other sensitive data. 2. It contains secrets (DB passwords, API keys). 3. `php artisan make:migration create_table_name` 4. `public/` - only this folder should be web-accessible. 5. `migrate` runs pending migrations; `migrate:fresh` drops all tables and re-runs everything.

Next Module

-> Module 01: PHP Fundamentals